INASS

International Journal of
Intelligent Engineering & Systems

http://www.inass.org/

Adaptive Web Penetration Testing with Q-Learning and Deep Q-Networks on OWASP Juice Shop

Kevin Imanuel Setiawan

Bachelor's Program in Information Technology, Telkom University, Surabaya, Indonesia

* Corresponding author: kevinit@student.telkomuniversity.ac.id

Abstract: This study compares tabular Q-learning (QL) and a deep Q-network (DQN) for adaptive web penetration testing on OWASP Juice Shop. Both agents operated for 1,585 episodes over 120 predefined endpoint-payload actions. Candidate findings passed control-differential verification, while cross-site scripting candidates also required browser execution. QL produced 14 verified action-level findings and DQN produced 13, across five unique endpoint-class locations each. Both agents found their first verified action in episode 1; QL reached its final observed count by episode 787 and DQN by episode 443. A separate detector benchmark yielded 13 true positives, no false positives, and one false negative among labeled actions; 46 actions remained unknown. A scoped two-endpoint comparison recorded 10 QL findings and one relevant ZAP alert, but the request procedures were not equivalent. The results demonstrate feasibility on this target and action set, not generalization to other applications.

Keywords: Web penetration testing, Reinforcement learning, Q-learning, Deep Q-network, OWASP Juice Shop, OWASP ZAP

1. Introduction

Web applications expose multiple input points and behaviors, so penetration testers must choose requests under limited time and request budgets. Reinforcement learning offers a way to learn action preferences from observed responses. Q-learning (QL) estimates action values in a table, whereas a deep Q-network (DQN) approximates values with a neural network [1]–[3]. Prior research has explored reinforcement learning for penetration testing and capture-the-flag environments, while emphasizing trade-offs between model-free learning and prior knowledge [4].

Earlier research frames penetration testing as a sequence of decisions in which observations guide the next probe. Model-free reinforcement learning is attractive because an agent can update action preferences through interaction without requiring a complete model of the target. Tabular Q-learning estimates action values directly for state-action pairs [1], [2], whereas deep Q-networks approximate action values with a neural network for richer representations [3]. Neither approach removes the need to define what actions are possible, how responses are represented, how rewards are assigned, and what evidence qualifies as a finding. Those design choices shape both the learned behavior and the claims that can be drawn from an experiment.

Research applying reinforcement learning to penetration testing and capture-the-flag tasks has highlighted the trade-off between model-free exploration and a priori knowledge [4]. A predefined endpoint and payload library improves repeatability and makes an experiment computationally manageable, but it narrows the search problem. This study declares the human-selected endpoints, payload classes, and endpoint surfaces as prior knowledge. The policy adapts its

selection among the predefined requests; it does not discover arbitrary URLs, crawl the application, generate new payloads, or conduct unrestricted autonomous testing.

The Agent Web Model formalizes web hacking as reinforcement-learning tasks at several abstraction levels and discusses the role of prior knowledge in defining those tasks [5]. Related work on cyber-terrain-aware attack graphs models operational constraints in the state and reward design [6]. Together, these studies highlight that an agent's observed performance depends on how the target, available actions, and constraints are represented. The present evaluation makes those boundaries explicit and adds candidate verification before a result is counted as a finding.

This study evaluates QL and DQN on OWASP Juice Shop, a deliberately vulnerable web application for security training [7]. We compare verified findings under a fixed episode budget and describe a scoped QL comparison with OWASP ZAP. Findings pass control-differential checks; XSS candidates additionally require browser execution. The evaluation is specific to the target and action set and does not establish generalization to arbitrary websites.

The evaluation addresses three descriptive questions: how many findings pass the verification gate for QL and DQN under the same episode budget; how their finding categories, endpoint activity, reward totals, and final-discovery episodes differ in the recorded runs; and what can be inferred from a two-endpoint comparison with ZAP when scan workflows remain different. The experiment does not test statistical significance or transfer to unseen applications.

2. Materials and Methods

2.1 Test Environment and Action Space

The target was OWASP Juice Shop version 20.2.0 running locally. Eight configured endpoints and 15 payloads formed 120 endpoint-payload actions: six SQL injection (SQLi), four cross-site scripting (XSS), and five fuzzing probes. Endpoint surface labels and payloads were manually curated from knowledge of the application. This human prior narrows the search space, so the system is not fully autonomous. The state comprises HTTP status, response length, error, reflection and vulnerability indicators, endpoint index, response-time category, and content type. DQN receives a 28-dimensional one-hot encoding.

The eight endpoint configurations cover product search, file access, product reviews, order tracking, user login, feedback, and challenge retrieval. GET probes are placed in the configured query or path; login, review updates, and feedback use structured request bodies. The feedback operation also obtains a captcha through the application endpoint and supplies required fields. These endpoint-specific request formats reduce the chance that malformed traffic is mistaken for evidence of security, while keeping the experiment within the implemented API flows.

Table 1. Configured endpoints and input surfaces.

Index	Method and path	Surface
0	GET /rest/products/search?q=	SQL
1	GET /ftp/	File
2	GET /rest/products/1/reviews	Plain
3	GET /rest/track-order/	Plain
4	POST /rest/user/login	SQL
5	PUT /rest/products/1/reviews	Plain
6	POST /api/Feedbacks	Plain
7	GET /api/Challenges	Plain

The action space is the Cartesian product of eight endpoint indices and 15 payload indices. An action index maps to endpoint = floor(action/15) and payload = action modulo 15. The 15 payload templates comprise six SQL injection probes, four cross-site scripting probes, and five fuzzing probes. The action set is fixed before training. The agent does not create new endpoints, infer unlisted routes, mutate payload syntax, or expand the payload library. It adapts only by choosing among these 120 predefined endpoint-payload pairs in response to the recorded state and reward.

The state summarizes the latest response and its request location. It contains HTTP status category, response-length category, error flag, reflection flag, vulnerability indicator, endpoint index, response-time category, and content-type category. The representation is not a full application model: separate server-side conditions may map to the same tuple if their recorded features coincide. For DQN, categorical fields are encoded as one-hot vectors: five status values, four response-length categories, eight endpoints, three response-time categories, and five content types. The three remaining indicators are scalar inputs. The

resulting DQN input dimension is 28; QL stores the discrete state tuple as a key.

Table 2. State variables and encodings.

Field	Encoding	Role
HTTP status	5 categories	Response class
Response length	4 categories	Size category
Error	Scalar flag	Request/server error evidence
Reflection	Scalar flag	Input-reflection evidence
Vulnerability	Scalar indicator	Analyzer classification
Endpoint	8 categories	Request location
Response time	3 categories	Latency category
Content type	5 categories	Response format

2.1.1 HTTP Response Preprocessing and State Construction

The HTTP client returns a decoded response object to the analyzer. For a successful response, the analyzer lowercases the decoded response text before pattern checks and calculates response length as the length of that text string. This is the implementation's character-count feature; it is not a measurement of the raw number of bytes transmitted on the network. A missing response is handled explicitly as a request error: the status code and response length are set to zero, the status category becomes error, and the vulnerability indicator is none. This prevents a timeout or connection failure from being interpreted as an exploitable server response.

Feature extraction then derives several signals from the response and the selected payload. The analyzer checks for database-error patterns, recognizes whether the payload belongs to a configured SQLi or XSS family, and tests whether the full payload or a partial reflection is present. Partial reflection is based on meaningful payload tokens of at least four characters; it is set when at least half of those tokens occur in the response text. It also records the response status class, elapsed-time class, content type, and whether the response differs significantly from the endpoint baseline.

For response length L and the endpoint-specific baseline B_e, a significant change is marked when $|L - B_e| / B_e \geq 0.15$, provided the baseline is positive. Baselines are collected before training by sending "hello" once to each of the eight endpoints. Using B_e rather than one global reference reduces false signals caused by endpoints that naturally return bodies of different sizes. The length categories used by the state and reward logic are split at 1,500, 2,500, and 3,500 characters: category 0 is below 1,500; category 1 is 1,500 to below 2,500; category 2 is 2,500 to below 3,500; category 3 is 3,500 or above.

The remaining categorical fields are derived consistently from configuration. HTTP status is grouped into 2xx, 3xx, 4xx, 5xx, or connection error. Response time is grouped as fast (<0.5 s), medium (0.5 s to below 2.0 s), or slow (≥2.0 s). Content type is classified into JSON, HTML, text, XML, or other. The eight components are combined into the discrete state tuple used by QL. DQN encodes the same categorical fields as 28 inputs: 5 status + 4 length + 1 error + 1 reflection + 1 vulnerability + 8 endpoint + 3 response-time + 5 content-type features. The preprocessing is feature engineering and discretization; it does not tokenize or vectorize the full response body.

2.2 Agent Configuration and Training Protocol

Each agent used 1,585 episodes and random seed 42. QL used learning rate 0.1, discount factor 0.9, epsilon from 1.0 to 0.1 with decay 0.90, and optimistic initial values of 5. DQN used a 64-unit hidden layer, learning rate 0.001, batch size 32, replay capacity 10,000, target update interval 10, and epsilon decay 0.9965. Exploration schedules differed to avoid DQN reaching its exploration floor too early. There was no early stopping. Only verified vulnerable actions were masked; previously attempted or failed actions remained available. A benign hello probe established a separate response baseline for each endpoint. Rewards were +50 for verified findings, +10 for anomalies or inconclusive candidates, −1 for normal responses, and −5 for errors, with a configured response-length bonus of 0, 0, 5, or 10.

Table 3. Agent hyperparameters.

Parameter	Q-learning	DQN
Episode budget	1,585	1,585
Learning rate	0.1	0.001
Discount factor	0.9	0.9
Exploration	ε: 1.0 to 0.1; decay 0.90	ε: 1.0 to 0.1; decay 0.9965
Value initialization	5.0	Neural network
Hidden units	—	64
Batch / replay	—	32 / 10,000
Target update	—	Every 10 updates
Random seed	42	42

The two methods use the same environment and observations but represent action values differently. QL updates a discrete table according to $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$, where s and s′ denote the current and next states, a is the selected action, r is its reward, α is the learning rate, and γ discounts future value [1], [2]. An epsilon-greedy policy samples an action

for exploration with probability ε and otherwise selects a high-valued action. Optimistic initialization assigns unvisited QL values a starting estimate of 5.0.

DQN approximates action values with a neural network and learns from stored transitions with a target network. The configured hidden dimension is 64, learning rate is 0.001, batch size is 32, replay capacity is 10,000, and target update interval is 10. The DQN epsilon decay of 0.9965 is more gradual than QL's 0.90. It was selected to preserve exploration longer under random initial network values. Thus, the comparison evaluates each algorithm with an appropriate configured schedule; it does not isolate network architecture while holding all hyperparameters numerically identical.

Reward is computed after candidate verification. The base outcomes and length-bonus categories are summarized in Table 4.

The reward function separates a request failure from the interpretation of a completed HTTP response. For a completed response, the base reward is +50 when the candidate vulnerability is verified, +10 when the analyzer reports an anomaly or a candidate is rejected/inconclusive, and −1 when no anomaly is present. The configured response-length bonus is then added to those three base outcomes. A connection error or timeout takes precedence and receives −5 without a length bonus.

The implemented rule can be written as $R_t = -5$ for a request error; otherwise $R_t = r_{base} + b(c_L)$, where $r_{base} \in \{+50, +10, -1\}$ and $b(c_L)$ is the bonus for response-length category c_L. The bonus vector is [0, 0, 5, 10] for categories 0–3. Thus, a verified finding in category 3 receives +60; an unverified candidate in category 2 receives +15; a normal category-3 response receives +9; and a request error remains −5. These examples clarify that a large response can earn a positive shaping bonus even without a verified vulnerability.

The reward is a learning signal, not a vulnerability metric. Cumulative reward is the sum of the per-episode rewards over the fixed 1,585-episode run and can therefore be affected by normal responses, errors, anomalies, and length bonuses as well as verified findings. A greater cumulative reward does not by itself mean that an agent found more distinct vulnerabilities. The experiment consequently reports reward alongside verified-finding counts, hit rate, discovery episode, and request cost.

Table 4. Reward components.

Outcome	Reward	Purpose
Verified vulnerability	+50	Positive signal after verification
Anomaly/inconclusive	+10	Signal without verified-vulnerability claim
Normal response	−1	Small negative signal
Error response	−5	Penalty for failed request
Length bonus	0, 0, +5, +10	Category-based additional reward

The system workflow is summarized in Fig. 1. The training protocol follows a repeatable sequence. Before training, the client sends the benign value "hello" to each endpoint and records a separate baseline. During each episode the policy selects one action, the HTTP client sends it, and the response analyzer generates the next state and candidate label. Any candidate then passes through its applicable verification gate. The final verified, anomaly, normal, or error result determines the reward; the transition is recorded and the agent updates its values before the next episode. This order separates baseline collection, detection, verification, and learning.

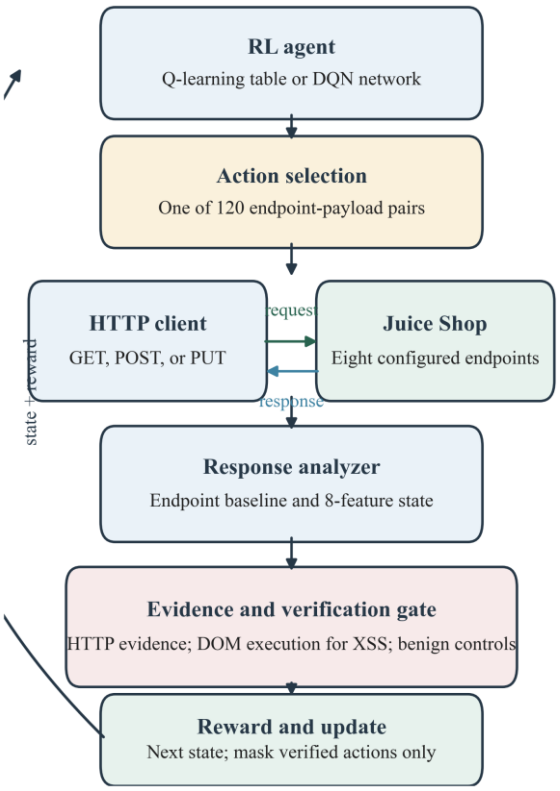


Fig. 1. System workflow for reinforcement-learning-based web testing.

Action masking applies only to actions already confirmed as vulnerable. Failed, normal, previously attempted, or rejected actions remain selectable. There is no exhausted-action masking and no early stop after the agent has encountered its currently known findings. Each agent therefore completes all 1,585 episodes. This fixed budget supports comparison of complete trajectories, although it also means requests continue after the final new finding in a run.

2.3 Finding Verification and Detector Benchmark

HTTP candidates were checked against a benign control and, when applicable, a neutralized-payload control; a finding was accepted only if neither reproduced its vulnerability class. Connection errors did not count as verification. XSS was evaluated in headless Chromium on the corresponding single-page application route; acceptance required execution of an instrumented canary, with a warmup navigation and bounded polling. A separate detector benchmark evaluated all 120 actions against curated labels. It used the browser path for XSS and excluded 46 unknown actions from binary metrics. Because ground-truth confirmation and detector evidence partly share mechanisms, this is not a fully independent oracle.

The analyzer also applied endpoint-surface constraints when assigning vulnerability classes. SQLi evidence required a database-error response or a sufficiently changed successful response relative to the endpoint baseline. File-disclosure labels were restricted to the configured file surface and required recognizable content or the configured extension-bypass evidence. Fuzzing findings required an echoed probe in a server-error response. These constraints reduce generic error-page and reflection-only findings, but their accuracy remains bounded by the configured signatures and curated surface labels.

The control-differential step addresses false positives by asking whether a benign request or a less exploit-like variant produces the same evidence. Payloads for which neutralization would make no meaningful change used only the benign control. An inconclusive candidate was retained as an anomaly reward rather than counted as a verified vulnerability. The browser check serves a separate purpose: it distinguishes data reflected in an HTTP response from a script that actually executes in the rendered application.

2.4 ZAP Comparison and Metrics

The controlled ZAP comparison was restricted to the two endpoints shared with QL: product search and the FTP surface. ZAP provides automated scanning and alert reporting [8]. We report verified finding count, hit rate (findings divided by training episodes), payload requests per finding, first-finding episode, episode at which the final observed finding count was reached, and unique endpoint–class locations. These are in-sample training results: no held-out test set was used. The endpoint scope is shared, but scan strategy, payload selection, verification overhead, and request accounting differ between QL and ZAP.

2.4.1 ZAP Architecture and Controlled Scan Flow

OWASP ZAP is organized around a local intercepting proxy and a set of core services and add-ons. The proxy/core records HTTP messages and maintains the site tree and session state; the Core API exposes actions and views to an external controller. ZAP's spider discovers links and resources, the active scanner sends test requests intended to probe for vulnerabilities, and the

passive scanner analyzes observed messages without modifying them. Findings from scanner rules are stored as alerts with attributes such as alert name, risk, URL, parameter, evidence, and attack. The controller can retrieve the message history and alerts through the API [9]–[12].

In this experiment, ZAP ran as a daemon at localhost:8080 and the Juice Shop target ran at localhost:3000. The Python controller first checked the ZAP version and counted existing target messages to establish a session baseline. For each of two allowed URLs—product search and /ftp/—it called the Core API to access the URL, started the traditional spider with maxChildren=2, polled the spider status until completion, then started an active scan with recurse=false and polled that scan to completion. The calls were made sequentially for the two URLs. No custom authenticated user or separate spider-wide target was configured by the controller. The controlled-scan report identifies ZAP version 2.17 and records 590 messages: 22 from URL access and spidering, and 568 from active scanning. It contains 33 raw alerts, of which one High/Medium SQLi alert passes the study's comparison filter.

After each phase, the controller queried the ZAP message history and subtracted the prior count to estimate messages generated by URL access/spidering and by active scanning. It then retrieved alerts through the Core API and classified them using the alert name or reference, mapping them to SQLi, XSS, path traversal, or other. Only alerts in the relevant vulnerability classes with High or Medium risk were included in the comparison. This post-processing step is the study's counting rule; it does not imply that excluded alerts are invalid for other security purposes.

Figure 2 summarizes this flow. It distinguishes the ZAP engine's automated scanning components from the project's external controller and alert filter. ZAP's passive scanner may process traffic recorded during exploration, but the controlled script's explicit scan stages are URL access, spider, and active scan; the script does not run a separate passive-scan job. The active scanner is the component that generates attack probes. This distinction matters when explaining the baseline and when comparing request counts with the RL agent, whose payloads are chosen one action at a time by its policy.

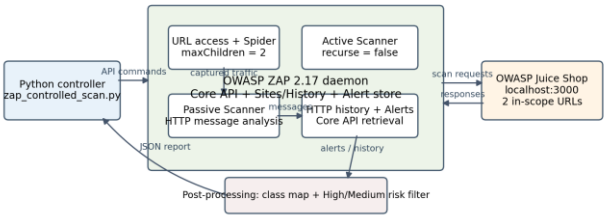


Fig. 2. Architecture and request flow of the controlled ZAP scan.

A finding count and a unique vulnerable location are also different measures. The unique-location measure counts distinct endpoint-index and vulnerability-class pairs. Several accepted payload-level findings at the same endpoint and class can contribute multiple findings but one location. This distinction prevents payload variants from being presented as separate vulnerable application locations.

The 1,585 episodes are the learning budget for each agent. The experiment also records baseline probes, verification probes, and total requests including verification. Episode count, selected payload-request count, and end-to-end HTTP transaction count therefore represent different quantities. Request-per-finding ratios must be read with their counting rule, particularly because control probes are issued only after candidates are detected.

3. Results and Discussion

3.1 QL and DQN Training Results

QL produced 14 verified action-level findings and DQN produced 13 (Table 5). Both found a vulnerability in episode 1. QL reached its final observed count by episode 787, whereas DQN did so by episode 443. These values describe accumulation during the recorded run and do not establish convergence or unseen-target performance. QL had more findings, while DQN accumulated a larger total reward, which reflects reward across all episodes rather than finding count alone.

Table 5. Training outcomes.

Measure	QL	DQN
Episodes	1,585	1,585
Verified findings	14	13
Categories	6 SQLi; 5 XSS; 3 fuzz/file	6 SQLi; 5 XSS; 2 fuzz/file
Hit rate	0.88%	0.82%
Requests/finding	113.21	121.92

First finding	Episode 1	Episode 1
Final count reached	Episode 787	Episode 443
Unique locations	5	5
Elapsed time	143.64 s	145.81 s
Total reward	3,783	13,033
Total transactions incl. baseline/verification	1,640	1,637

The category counts show six SQLi and five XSS action-level findings for each agent. QL reported one additional fuzzing/file finding, while both agents reported five XSS findings. Multiple payload findings at a single endpoint and class contribute one unique location; these results are not an inventory of independent production vulnerabilities.

Both agents reported six SQLi findings. QL's one-finding advantage is accounted for by the same number of XSS findings and one additional fuzzing/file finding. The counts do not measure severity, exploitability in a production context, or remediation priority; the experiment reports verified class outcomes rather than a risk-weighted score. Fig. 4 visualizes category counts to make this composition difference visible without implying that the categories have equal operational impact.

The first finding appeared at episode 1 for both agents, so this measure does not distinguish their initial discovery in the recorded runs. DQN reached its own final finding count by episode 443, before QL reached its final count by episode 787. This does not mean DQN found more issues: the count measures total accepted findings, whereas the episode measures when the run reached that count. Figure 3 shows the cumulative unique verified-action curve for each run. A single seeded trajectory cannot show whether either pattern is stable across runs.

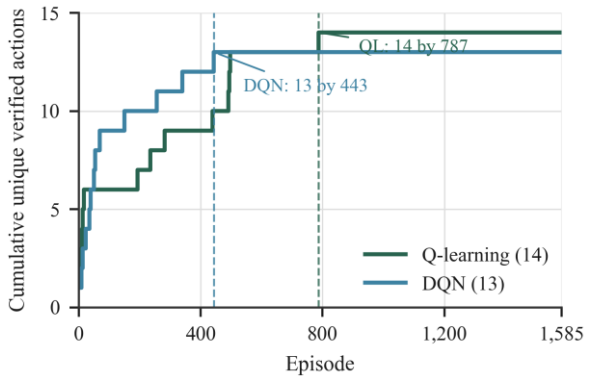


Fig. 3. Cumulative unique verified-action discoveries by episode.

The cumulative reward should not be interpreted as a direct vulnerability-coverage metric. It aggregates rewards for every episode, including anomalies, normal responses, errors, and length bonuses. DQN can therefore earn a higher total reward even while reporting fewer verified findings. Future comparisons should report reward and verified-finding curves together and summarize variation over repeated seeds.

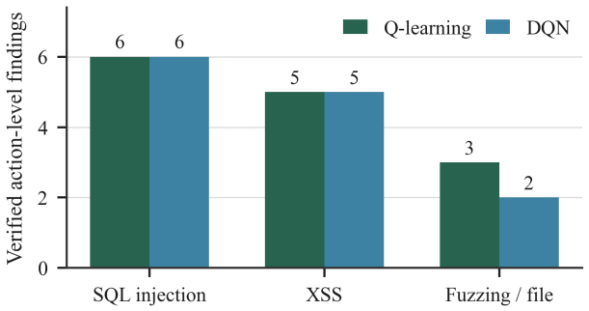


Fig. 4. Verified action-level findings by category.

3.2 QL Endpoint Activity

Product search accounted for most QL requests and findings. The FTP surface produced three findings from 41 requests; track-order produced three; login produced one. Other configured endpoints produced no verified findings in this run. The distribution reflects the curated action space and should not be interpreted as a complete vulnerability inventory.

The endpoint request counts sum to the 1,585 episode actions. Product search accounts for 1,173 requests, or roughly three quarters of the budget, while track-order accounts for 245. The other six endpoints together account for 149 requests. The learned policy therefore concentrated its actions at search and track-order, especially search. This describes policy allocation in the run; it does not demonstrate that the remaining endpoints were exhaustively tested outside the configured payload set.

The endpoint finding counts sum to 14: seven for search, three for FTP, three for track-order, and one for login. The report separately gives five unique vulnerable locations because multiple accepted payload findings can correspond to the same endpoint-class pair. Alert consolidation rules vary between tools, so any cross-tool comparison must state whether it counts payload-level findings, unique locations, or consolidated alerts.

The difference between reward and verified-finding totals follows from the reward design. A vulnerability contributes a large positive reward, but the final cumulative score also includes normal responses, errors, anomalies, and response-length bonuses throughout all episodes. DQN can therefore obtain a higher cumulative reward while producing fewer distinct accepted findings. Reward should be interpreted as an optimization signal used by each agent, not as a direct substitute for vulnerability coverage.

Table 6. QL requests and verified findings by endpoint.

Endpoint	Requests	Findings
Product search	1,173	7
FTP surface	41	3
Product reviews (GET)	39	0
Track order	245	3
User login	24	1
Product reviews (PUT)	21	0
Feedback	22	0
Challenges	20	0
Total	1,585	14

3.3 Detector Benchmark

The labeled benchmark contained 13 true positives, no false positives, one false negative, and 60 true negatives; 46 actions remained unknown. Precision was 1.000, recall 0.929, F1 0.963, and specificity 1.000. The false negative was the SVG XSS action on the track-order route. Zero false positives applies only to the 60 labeled negatives and does not establish perfect performance on untested applications or payloads.

Precision is TP/(TP+FP), recall is TP/(TP+FN), F1 is the harmonic mean of precision and recall, and specificity is TN/(TN+FP). These values are computed only for labeled actions. The 46 unknown actions are not assumed safe and are excluded from the negative class; recoding them as negatives would inflate specificity without evidence. The one false negative, an SVG XSS action on the track-order route, also shows that the configured detector and verification path did not recover every curated positive.

No false positives among 60 labeled negative actions is useful evidence that the gates suppressed several noisy response patterns in this benchmark. It is not a guarantee of zero false positives in future tests: the labeled set is finite, labels are curated, and the oracle partly shares the detector's evidence mechanisms. A stronger independent assessment would have blinded reviewers adjudicate browser and server evidence without seeing detector output or agent identity.

Table 7. Detector performance on labeled actions.

Metric	Value
True positives	13
False positives	0
False negatives	1
True negatives	60
Unknown actions	46
Precision	1.000
Recall	0.929
F1-score	0.963
Specificity	1.000

3.4 Scoped Comparison with OWASP ZAP

For the two shared endpoints, the QL run log records 1,214 QL payload requests for 10 findings (121.40 requests per finding). The controlled ZAP scan reports 590 requests and one relevant finding (590.00 requests per finding). The ratio of these rates is approximately 4.86. The current controlled ZAP report contains 33 raw alerts; one High/Medium SQLi alert meets the comparison rule, while no XSS or path-traversal alert does. This is endpoint-scope matching, not a fully procedure-matched apples-to-apples comparison: request workflow, payloads, verification overhead, and counting rules differ. The 4.86 ratio is descriptive for this experiment and cannot support a general efficiency claim.

Restricting both tools to product search and FTP makes the target scope more comparable than comparing QL's eight-endpoint run with a broader scan. Still, QL selects individual payload actions through its policy, while ZAP follows its own spidering and active-scan workflow. They can issue different request types and volumes, apply different payloads, and use different alert-consolidation and acceptance criteria. QL verification probes and the ZAP workflow also contribute differently to the request totals.

The request-per-finding ratio of approximately 4.86 is descriptive rather than a normalized effect estimate. It does not control for scan duration, payload complexity, severity, manual review cost, or the different baseline and verification transactions. The raw request and finding counts should be reported beside the ratio. A stronger comparison would align the endpoint allowlist, authentication state, request and time budgets, payload families, request accounting, alert definitions, and independent manual confirmation.

Table 8. Scoped comparison on two shared endpoints.

Method	Requests	Relevant findings	Requests/finding
QL selected-payload subset	1,214	10	121.40
Controlled ZAP scan	590	1	590.00

The ratio 590.00/121.40 is approximately 4.86, but it is descriptive rather than a normalized effect estimate. It does not control for scan duration, payload complexity, severity, manual review cost, or the different baseline and verification transactions. The raw request and finding counts should be reported beside the ratio. A stronger comparison would align the endpoint allowlist, authentication state, request and time budgets, payload families, request accounting, alert definitions, and independent manual confirmation.

The evaluation used one target and one seed per algorithm; training results are in-sample. Endpoint labels and payloads encode human knowledge. The detector benchmark shares evidence with its verification oracle, and the ZAP comparison does not align all scan procedures. Future work should repeat runs across seeds, use held-out targets or actions, obtain independent manual validation, and equalize request and time budgets for baseline comparisons.

3.5 Interpretation of the QL–DQN Difference

The finite action set contains 120 choices and the state representation is compact and engineered. Tabular QL is consequently a reasonable baseline because values can be stored for discrete observed states without function approximation. DQN may be useful when state representations grow or share structure, but this experiment does not establish that advantage. The finding totals and reward totals are outcomes of one run under different exploration schedules; they should be treated as an implementation comparison rather than a general ranking of learning algorithms.

3.6 Threats to Validity

Construct validity is limited by the operational definition of a finding: a candidate must match the analyzer rules and pass its verification gate. The evaluation does not assign severity or measure business impact. Internal validity is limited by a single random seed per agent and algorithm-specific hyperparameters. The observed differences may reflect trajectory randomness and configuration choices as well as the learning algorithm.

External validity is limited to one version of one intentionally vulnerable application, one local deployment, and one curated payload library. Endpoint surfaces and payload choices encode human knowledge. The test does not cover changing production state, real authentication workflows, rate limits, network variability, or a broad range of web frameworks. A fixed seed improves reproducibility but cannot estimate run-to-run variation.

Comparison validity is limited because ZAP and the RL agents use different scan workflows and request-count rules. Equal endpoint scope alone does not ensure equal opportunities to find vulnerabilities. Consolidation rules and evidence thresholds also affect the finding numerator. These limitations constrain the interpretation of the 4.86 ratio and preclude a general efficiency claim.

3.7 Follow-up Experiments

Future work should run each method across multiple seeds and report mean, spread, confidence intervals, and per-seed discovery curves. A held-out application or endpoint set would test transfer, provided the held-out labels are not used to design the action space or detection rules. Ablations could remove surface gating, browser verification, or optimistic initialization to estimate how each design choice affects discovery and false positives.

For a stronger ZAP comparison, both systems should use a published endpoint allowlist and aligned authentication state. Request budgets and wall-clock budgets should be reported separately, since equalizing only one can favor a system with more expensive verification. Candidate findings should be deduplicated by endpoint, vulnerability class, and evidence, then manually reviewed by an assessor blinded to tool identity. Coverage, request cost, time-to-first-verified-finding, and false-positive review burden can then be compared under a shared protocol.

4. Conclusion

Under 1,585 episodes on OWASP Juice Shop, QL obtained 14 verified action-level findings and DQN obtained 13. DQN reached its final observed finding count earlier, while accumulating greater total reward. The detector benchmark agreed strongly with curated labels, subject to partial dependence on shared verification evidence. The scoped ZAP comparison favored QL in requests per finding, but procedural differences make that result descriptive. Repeated and held-out

evaluations are required before drawing broader conclusions.

Conflict of Interest

The author declares that there is no conflict of interest.

References

[1] C. J. C. H. Watkins and P. Dayan, “Q-learning,” Machine Learning, Vol. 8, No. 3–4, pp. 279–292, 1992, doi: 10.1007/BF00992698.

[2] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed., MIT Press, Cambridge, MA, 2018.

[3] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, Vol. 518, pp. 529–533, 2015, doi: 10.1038/nature14236.

[4] F. M. Zennaro and L. Erdodi, “Modeling penetration testing with reinforcement learning using capture-the-flag challenges: Trade-offs between model-free learning and a priori knowledge,” arXiv:2005.12632, 2020.

[5] L. Erdodi and F. M. Zennaro, “The Agent Web Model: Modelling web hacking for reinforcement learning,” arXiv:2009.11274, 2020.

[6] R. Gangupantulu, T. Cody, P. Park, A. Rahman, L. Eisenbeiser, D. Radke, and R. Clark, “Using cyber terrain in reinforcement learning for penetration testing,” arXiv:2108.07124, 2021.

[7] OWASP Foundation, “OWASP Juice Shop,” [Online]. Available: https://owasp.org/projects/juice-shop?tab=overview. Accessed: Oct. 12, 2026.

[8] OWASP ZAP, “Automation Framework,” [Online]. Available: https://www.zaproxy.org/docs/automate/automation-framework/. Accessed: Oct. 12, 2026.

[9] OWASP ZAP, “Getting Started,” [Online]. Available: https://www.zaproxy.org/docs/desktop/start/. Accessed: Oct. 12, 2026.

[10] OWASP ZAP, “API Reference,” [Online]. Available: https://www.zaproxy.org/docs/api/. Accessed: Oct. 12, 2026.

[11] OWASP ZAP, “Passive Scanner Add-on,” [Online]. Available: https://www.zaproxy.org/docs/desktop/addons/passive-scanner/. Accessed: Oct. 12, 2026.

[12] OWASP ZAP, “Automation Framework: Active Scan Job,” [Online]. Available: https://www.zaproxy.org/docs/desktop/addons/automation-framework/job-ascan/. Accessed: Oct. 12, 2026.